

When is a Payment needed?

Twenty-two use cases for the durable collection object, the one question that decides whether you need it, and which resource a client uses depending on whether they already run an order management system.

Payment = durable collection obligation
Attempt = a particular execution of that payment
the single most important semantic distinction

The test

Is the intent to collect independent of any single attempt to execute it?

Internally

Always created

Externally

Progressively disclosed

Contents

1. The question that defines Payment	3
2. The use-case catalogue	3
3. Why Payment exists even when it is invisible	5
4. Which resource the client actually uses	5
5. Authorization is not Capture	12
6. Multiple attempts	13
7. Authentication	13
8. Omnichannel	14
9. Multi-tender	15
10. Subscription	15
11. Invoice	16
12. Refunds and disputes	17
13. Agentic commerce	18
14. When Payment need not be exposed	19
15. The architecture to lock in	20

SECTION 1

The question that defines Payment

Payment is needed whenever the merchant needs a durable money-collection commitment that can survive multiple interactions, attempts or asynchronous steps.

THE TEST

Does the merchant need to maintain an ongoing intent to collect this particular amount of money, independent of any single attempt to execute it?

If yes → Payment. If no → an Attempt may be enough.

1.1 The common misunderstanding

WHAT DOES NOT REQUIRE A PAYMENT

You do **not** need a Payment object merely because you are processing a card authorization. You need one when there is value in maintaining payment state **independently from the individual execution attempt**.

1.2 The distinction in one line

Payment = what you are trying to collect
Attempt = how you tried to collect it

That separation is what allows one payment model to serve e-commerce, recurring, invoices, multi-tender, omnichannel, card present, asynchronous rails, retries, routing, marketplaces, franchises and agentic commerce — without creating a different payment model for every use case.

SECTION 2

The use-case catalogue

Twenty-two cases where a durable Payment object is required, and why. Every one of them is available to every client — Section 4 covers which resource they use to reach it.

USE CASE	WHY PAYMENT IS NEEDED	TYPICAL LIFECYCLE
Standard e-commerce checkout	Payment must survive authentication, retries and redirects	Create → Confirm → Authorize → Capture

USE CASE	WHY PAYMENT IS NEEDED	TYPICAL LIFECYCLE
Authorization now, capture later	Authorization and capture happen at different times	Create → Authorize → Capture
Partial capture	Multiple captures against one authorization	Authorize → Capture \$60 → Capture \$40
Incremental authorization	Hotel, rental or fuel needs to increase the authorized amount	\$500 auth → +\$200 increment
Delayed fulfilment	Merchant needs a payment commitment before goods or services are delivered	Payment → Fulfilment → Capture
Asynchronous payment method	Bank transfer, ACH or open banking may complete later	Created → Processing → Succeeded
3DS and customer authentication	Payment persists while the customer completes authentication	Requires action → Processing
Payment retry	The same commercial payment can have multiple attempts	Attempt 1 decline → Attempt 2 success
Smart routing	Payment can be replanned across processors and acquirers	Attempt A → retry, route B
Multi-tender checkout	Order needs multiple independent payment commitments	Order → Payment 1 + 2 + 3
Omnichannel	Payment survives movement between channels	Web → Store → Terminal
Payment link	The link or session may expire, but the underlying payment remains	Payment → Link → Customer pays
Terminal payment	A terminal attempt can fail while the payment remains open	Payment → Attempt 1 → Attempt 2
Wallet or method selection	Customer can change credential without creating a new commercial payment	Payment → PM A → PM B
Stored credential and vault	Payment references a durable credential and may need retries	Payment → Credential → Attempt
Recurring billing	Invoice needs a durable payment collection object	Invoice → Payment → Attempts
Invoice payment	Invoice can remain unpaid while collection attempts occur	Invoice → Payment → Retry
Marketplace and platform	Payment must remain independent of allocation and payout	Payment → Obligations → Payouts
Franchise	One payment may create obligations to multiple parties	Payment → Allocation → Settlement
Agentic commerce	Agent authorization, mandate and execution may be separated in time	Delegation → Payment → Attempt
Buy online, pick up in store	Payment exists before physical fulfilment	Online payment → Store fulfilment
Buy online, return in store	Original payment must remain identifiable for the refund	Original payment → CP refund

SECTION 3

Why Payment exists even when it is invisible

A simple card sale could technically be executed without a durable Payment object. We create one anyway.

Customer, merchant, card, authorization, capture. Nothing about that flow demands a durable object, and a client integrating at the simplest level will never name one.

3.1 What the Payment gives you regardless

Creating it internally provides a **stable identity** for everything that comes afterwards:

STABLE IDENTITY FOR	STABLE IDENTITY FOR
Authorization	Customer servicing
Capture	Events
Refund	Analytics
Dispute	Retries
Reconciliation	Omnichannel continuity

THE RESULTING PRINCIPLE

The merchant does not necessarily need to see that complexity. Payment is a first-class canonical object *internally*, and progressively disclosed *externally*. Sections 4 and 14 set out who sees it and when it stays hidden.

SECTION 4

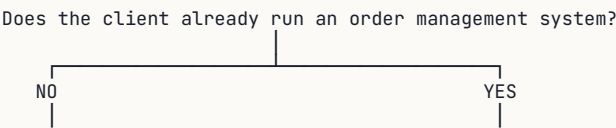
Which resource the client actually uses

Every use case in Section 2 exists on the platform. Which resource the client touches to reach it depends on one thing about the client — not on the use case.

THE BRANCH

Does the client already run an order management system?

If no → they use the **Order** resource, and Order abstracts Payment for them. If yes → they use **Payment** or **Attempt** directly, passing their own order ID as a reference.



Use the ORDER resource.
Order abstracts Payment
for them.

↓

We hold the order record.
We create Payment, Session
and Attempt internally.

Use PAYMENT or ATTEMPT.
Pass your own order ID
as a reference.

↓

Their OMS holds the record.
We create Payment and Attempt
and correlate by reference.

4.1 Clients without an order management system

These clients have no system of record for what the customer is buying. The Order resource becomes that record, and it **abstracts Payment away entirely**.

```
POST /orders
POST /orders/{id}/checkout

→ checkout_url
```



The client never names a Payment. Everything in Section 2 — delayed capture, retries, authentication, multi-tender, omnichannel continuity — still happens, driven by the Order and by platform policy.

What the Order carries for them

Because the Order is their system of record, it must hold everything an order management system would otherwise hold — not just an amount.

COMPONENT	CONTAINS	WHY AN OMS-LESS CLIENT NEEDS IT
Line items	Reference, description, quantity, unit amount, tax, discount, fulfilment mode, fulfilled and returned quantity	The record of what was bought. Drives partial fulfilment, partial return and Level 2/3 data.
Amounts	Subtotal, tax, shipping, discount, total owed	One authoritative statement of what the customer owes.
Tax	Jurisdiction, rate, amount per line, exemption status	Needed for invoices, receipts and marketplace facilitator remittance.
Customer	Customer reference, contact, billing and shipping address	Receipts, AVS, dispute evidence, returns without a receipt.
Fulfilments	Mode, location, carrier, tracking, dates, per-line quantities	Drives capture timing, the collectibility gate and delivery evidence.
Returns	Returned lines and quantities, reason, exchange flag	Distinguishes value returned in an exchange from money actually refunded.
Tender lines	Type, amount, payment reference, and externally-settled tenders	Makes split tender and proportional refunds computable.

COMPONENT	CONTAINS	WHY AN OMS-LESS CLIENT NEEDS IT
Payments	References to every Payment against the order	The join between what is owed and how it is being collected.
Obligations	Beneficiary, basis, release condition, amount	Marketplace, franchise, platform fee and tax remittance claims.
Ledger link	Owed, collected, refunded, open, and the ledger transactions behind them	Financial truth for the order, derived rather than asserted.
Status	Commercial state, derived financial state, derived collectibility	Three independent answers that must never be collapsed into one field.
Discounts and promotions	Applied at line or order level, with reason	Revenue recognition and refund arithmetic both need the discount attributed correctly.
Metadata	Client key-value pairs	Their own references, never read by platform logic.

The Order object

```
{
  "id": "ord_1Pm4KqYd8vRnT2Bx",
  "status": "OPEN",
  "financial_status": "PARTLY_COLLECTED",
  "collectible": true,
  "currency": "USD",
  "customer": "cust_1Pn3RqKd7wXtY2Mz",

  "amounts": {
    "subtotal": 30300, "tax": 2500, "shipping": 1200,
    "discount": 1800, "total_owed": 32200
  },

  "line_items": [
    { "id": "li_9Kd2mXpQr4Wt", "reference": "SKU-88213-BLK-M",
      "description": "Merino Crew Sweater, Black, M",
      "quantity": 2, "unit_amount": 8900, "tax": 1469,
      "fulfilment_mode": "shipment",
      "fulfilled_quantity": 2, "returned_quantity": 0,
      "beneficiary": "ca_1PmXqLdY8vKrN3Bt" }
  ],

  "fulfilments": [
    { "id": "ful_8Nq3WdKm2Rt", "location": "loc_118",
      "carrier": "ground", "tracking": "1Z999AA10123456784",
      "shipped_at": 1755993614, "lines": [ { "line": "li_9Kd2mXpQr4Wt", "quantity": 2 } ] }
  ],

  "returns": [],

  "tender_lines": [
    { "type": "card", "amount": 19000, "payment": "pay_9f186d77" },
    { "type": "gift_card", "amount": 5000, "external": true }
  ],

  "payments": [ "pay_9f186d77" ],
  "obligations": [ "obl_5Tn8QwEr2Zx", "obl_2Hj6Df6h4Rt" ],

  "ledger": {
    "owed": 32200, "collected": 24000, "refunded": 0, "open": 8200
  }
}
```

```

},

"metadata": { "campaign": "fall-merino" }
}

```

THE DESIGN CONSEQUENCE

Everything above exists so the client does not have to. A merchant with no order system gets line items, tax, fulfilment, returns, tender lines, obligations and a ledger **without building any of it** — and Payment stays abstracted beneath. A merchant who already has all of this keeps theirs and passes `order_ref` instead.

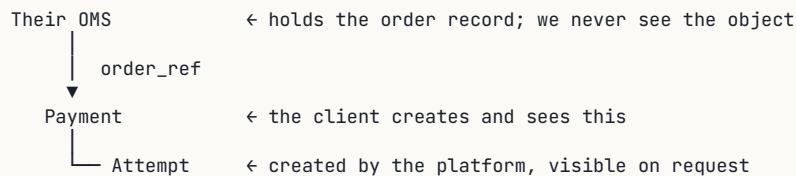
4.2 Clients with an order management system

These clients already have a system of record and will not create a second order anywhere. **No Order resource is created on our side.** They pass their own order identifier as a reference so that money and commerce can be joined.

```

POST /payments
{
  "amount": 32200,
  "currency": "USD",
  "order_ref": "WEB-2026-0884213",    ← their order ID, from their OMS
  "payment_method": "pm_123",
  "confirm": true
}

```



Two depths within this branch

THEY WORK AT	WHEN	WHAT THEY GET
Payment	They need a durable collection commitment — delayed capture, retries, authentication, channel continuity	The full lifecycle: confirm, capture, refund, and a stable identity for disputes and reconciliation
Attempt	They want to observe or reason about individual executions	Every execution with its route, credential, response code and outcome. Read-mostly; the Payment still governs

4.3 The rule that prevents a common error

THE BRANCH IS A PROPERTY OF THE CLIENT

It is decided **once**, at onboarding, and applies to every channel that client operates. An enterprise retailer is on the same branch for its website, its stores, its call centre and its kiosks. A small merchant is on the same branch for its online store and its counter.

The channel never changes the branch, and neither does the use case. Assigning a branch per use case would mean one retailer needing two integrations and two sources of truth for the same customer.

4.4 What each branch sees

OBJECT	NO OMS	HAS OMS
Order record	Ours — they create it	Theirs — we never hold it
Order identifier	<code>ord_</code> from us	Their own, passed as <code>order_ref</code>
Payment	Created for them, usually unnamed	Created by them, named and controlled
Session	Created for them	Created by them when they need one
Attempt	Internal	Visible on request
Capture, refund, dispute	Resolved through the Order	Resolved through the Payment

WHY THIS DOES NOT CHANGE THE MODEL

Both branches run on the same objects, the same state machines and the same decision engine. What differs is only **how much the client has to name**. This is the same progressive-disclosure principle as Section 14, applied one level higher — at the commerce layer rather than the payment layer.

4.5 What the Order resource has to contain

For a client with no order management system, the Order **is** their order system. It therefore has to carry everything such a system would, not merely enough to take a payment.

GROUP	HOLDS	WHY A CLIENT WITHOUT AN OMS NEEDS IT
Identity	<code>id</code> , <code>reference</code> , <code>status</code> , <code>created</code> , <code>currency</code> , <code>channel</code>	A stable handle for the customer, support and reporting.
Customer	<code>customer</code> , contact, billing and shipping addresses	Who bought it and where it goes. Also feeds address verification.
Line items	SKU, description, quantity, unit amount, tax, discount, beneficiary, fulfilment mode	What was bought. Drives tax, fulfilment, returns and interchange data.
Amounts	subtotal, tax, shipping, discount, total	What is owed, broken down so it can be amended and audited.
Ledger	owed, collected, refunded, open, tender lines	The running position across every payment and every tender.
Payments	The payments collecting against this order	Split tender, deposits and balances without the client doing arithmetic.

GROUP	HOLDS	WHY A CLIENT WITHOUT AN OMS NEEDS IT
Obligations	Who is owed what, on what basis, released when	Marketplaces, franchises, platform fees and tax remittance.
Fulfilments	Shipments, pickups, deliveries, carrier and tracking	Drives capture timing, collectibility and dispute evidence.
Returns	Returned quantities, reason, restocking, exchange flag	Drives refund resolution across tender lines.
Documents	Invoice, receipt, packing slip	Clients without an OMS have nowhere else to generate these.
Timeline	Every event on the order, in order	Customer service and dispute evidence in one place.
Metadata	Arbitrary key-value pairs	Client-specific context the platform never interprets.

4.6 Sub-resources

```

ORDER  ord_
├── line_items[]    li_    what was bought
├── tender_lines[]  tl_    how it was paid, including tenders we did not process
├── payments[]      pay_    collection against this order
├── obligations[]   obl_    who is owed what
├── fulfillments[]  ful_    shipments, pickups, deliveries
├── returns[]       ret_    what came back
├── documents[]     doc_    invoice, receipt, packing slip
└── events[]        evt_    the timeline

```

```

DERIVED, never written by the client:
  amounts.total    from line items, tax, shipping and discount
  ledger.*         from payments, refunds and tender lines
  status           from fulfillments and returns
  financial_status from the ledger
  collectible      from payment state, expiry horizon and risk holds

```

THE DIVISION THAT KEEPS THIS SAFE

The client writes **what was bought and what happened to it** — line items, fulfillments, returns. The platform derives **everything about money** — totals, ledger, financial status, collectibility. A client can never put the order into a state where the money does not add up, because they cannot write those fields.

4.7 A full Order

A furniture retailer with no order management system. Two items, one shipped and one on backorder, paid with a gift card and a card.

```
{
  "id": "ord_1Pm4KqYd8vRnT2Bx",
  "object": "order",
  "reference": "S0-2026-0471",
  "created": 1757462400,
  "currency": "USD",
  "channel": "ecommerce",
  "status": "partially_fulfilled",
  "financial_status": "collected",
  "collectible": true,

  "customer": {
    "id": "cust_1Pn3RqKd7wXtY2Mz",
    "email": "j.okafor@example.com",
    "phone": "+1-214-555-0143",
    "billing_address": { "line1": "4200 Legacy Dr", "city": "Frisco",
      "state": "TX", "postal_code": "75034", "country": "US" },
    "shipping_address": { "line1": "4200 Legacy Dr", "city": "Frisco",
      "state": "TX", "postal_code": "75034", "country": "US" }
  },

  "line_items": [
    { "id": "li_9Kd2mXpQr4Wt", "sku": "SOFA-LNN-3S-GRY",
      "description": "Linen 3-seat sofa, grey", "quantity": 1,
      "unit_amount": 129900, "discount": 10000, "tax": 9892,
      "amount_total": 129792, "fulfillment_mode": "delivery",
      "fulfilled_quantity": 1, "returned_quantity": 0 },
    { "id": "li_3Rt7NvBc8Ykm", "sku": "OTTO-LNN-GRY",
      "description": "Matching ottoman, grey", "quantity": 2,
      "unit_amount": 24900, "discount": 0, "tax": 4109,
      "amount_total": 53909, "fulfillment_mode": "delivery",
      "fulfilled_quantity": 0, "returned_quantity": 0 }
  ],

  "amounts": {
    "subtotal": 179700, "discount": 10000, "tax": 14001,
    "shipping": 0, "total": 183701
  },

  "ledger": {
    "owed": 183701, "collected": 183701, "refunded": 0, "open": 0,
    "tender_lines": [
      { "id": "tl_5Tn8QwEr2Zx", "type": "gift_card", "amount": 25000,
        "processed_by_platform": false, "reference": "GC-88213" },
      { "id": "tl_7Vb3MkLp9Ac", "type": "card", "amount": 158701,
        "processed_by_platform": true, "payment": "pay_9f186d77cb1b" }
    ]
  },

  "payments": [
    { "id": "pay_9f186d77cb1b", "amount": 158701, "status": "partially_captured",
      "captured": 129792, "remaining_capturable": 28909 }
  ],

  "obligations": [
    { "id": "obl_2Hj6DfGh4Rt", "party": { "type": "merchant",
      "id": "acct_4Nm8QdKt2Wp" }, "basis": "line_items",
      "gross_amount": 169700, "status": "open" },
    { "id": "obl_8Wq1ZxCv5Nb", "party": { "type": "tax_authority",
      "id": "US-TX-COLLIN" }, "basis": "declared_tax",
      "gross_amount": 14001, "status": "open",
```

```

    "release_condition": { "type": "filing_calendar", "schedule": "monthly_20th" } }
  ],
  "fulfillments": [
    { "id": "ful_8Nq3WdKm2Rt", "status": "delivered",
      "line_items": [ { "id": "li_9Kd2mXpQr4Wt", "quantity": 1 } ],
      "carrier": "regional_freight", "tracking": "RF-4471-88213",
      "delivered_at": 1758067200 }
  ],
  "returns": [],
  "documents": [
    { "id": "doc_4Kt9WmBd2Nv", "type": "receipt",
      "url": "https://docs.example.com/r/doc_4Kt9WmBd2Nv" }
  ],
  "metadata": { "showroom": "frisco-01", "associate": "emp_2291" }
}

```

What the client wrote, and what the platform derived

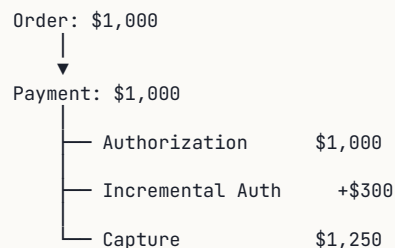
CLIENT WROTE	PLATFORM DERIVED
Line items, quantities, unit amounts, discounts	<code>amounts.*</code> , including the total
Customer and addresses	<code>ledger.*</code> — owed, collected, refunded, open
The fulfilment, when the sofa was delivered	<code>status</code> and <code>financial_status</code>
The externally-settled gift card tender	<code>collectible</code>
Metadata	<code>payments[]</code> , <code>obligations[]</code> and the capture that followed delivery

SECTION 5

Authorization is not Capture

Probably the strongest single reason the object must exist.

5.1 Worked example — a hotel stay



The Payment gives you a durable container for the whole authorization and capture lifecycle. The authorized amount rose, the captured amount differs from both the original authorization and the total authorized, and none of that required a second payment object.

5.2 Where this pattern appears

SEGMENT	SEGMENT
Hotels	Travel
Car rental	Delayed shipment
Lodging	Marketplace fulfilment
Restaurants	
Fuel	

SECTION 6

Multiple attempts

Another fundamental reason, and the one that matters most to the orchestration strategy.

Payment PAY-123
Amount: \$500

Attempt 1	Visa / Acquirer A	→ Declined
Attempt 2	Visa / Acquirer B	→ Declined
Attempt 3	Network Token / Acquirer C	→ Authorized

THE REQUIREMENT

You do not want three Payments.

You want one Payment with three Attempts beneath it.

Payment

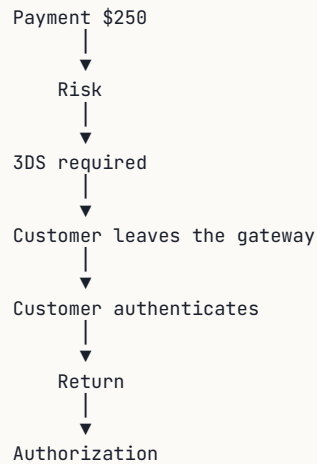
- Attempt 1
- Attempt 2
- Attempt 3

The Payment represents **what you are trying to collect**. The Attempt represents **how you tried to collect it**. That separation is what makes smart routing, replanning and retry orchestration expressible at all — without it, every route change is a new money object for the merchant to reconcile.

SECTION 7

Authentication

The payment must persist while the customer is somewhere else entirely.



THE RULE

You do not want the authentication interaction to create a new payment. The Payment persists across the entire flow, including the period where the customer is not on the merchant's site at all.

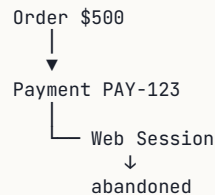
The same shape recurs for every redirect-based method, wallet handoff, open-banking consent and human-in-the-loop confirmation. One state serves all of them.

SECTION 8

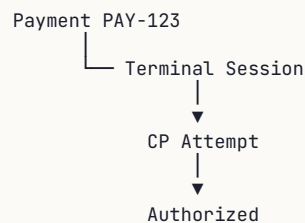
Omnichannel

One of the strongest arguments for the architecture.

The customer starts online



The customer walks into the store



WHAT THIS DEMONSTRATES

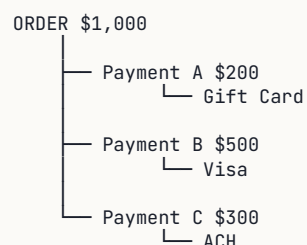
Same Payment. Different Session. Different Attempt.

This gives true channel independence — and it is only possible because the Payment is durable and the Session is not responsible for the money.

SECTION 9

Multi-tender

Each collection source has its own lifecycle, so each needs its own Payment.



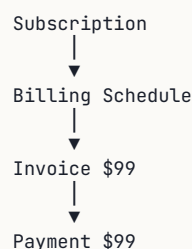
Independent Payment objects are not optional here. The gift card may settle instantly, the Visa may authorize and capture separately, and the ACH may take two days — three lifecycles that cannot share one object.

OBJECT	ANSWERS
The Order	What does the customer owe?
The Payments	How are we collecting it?

SECTION 10

Subscription

Where Payment becomes particularly important, because three objects are competing to be "the payment" and only one of them is.



```
Attempt 1 → Declined
Retry    → Declined
Retry    → Authorized
```

THE TWO THINGS THAT ARE NOT THE PAYMENT

The Subscription is not the Payment. It is the agreement.

The Invoice is not the Payment. It is what is owed for a period.

10.1 The chain, and who owns what

```
Subscription
  ↓
Invoice
  ↓
Payment
  ↓
Attempts
```

LAYER	RESPONSIBILITY
Billing	Determines <i>what</i> should be collected, and when it becomes due.
Payment	Determines the durable collection lifecycle.
Decision Engine	Determines <i>how</i> to collect it — route, credential, retry timing.

Dunning is therefore a conversation between billing and the decision engine, mediated by a Payment that outlives every individual attempt.

SECTION 11

Invoice

Another very strong case, and the one that forces multiple Payments against a single obligation.

11.1 Asynchronous settlement

```
Invoice INV-1001
Amount: $10,000
Due:    Sept 30
  ↓
Payment PAY-1001
  |
  |— ACH Attempt
  |   ↓
  |   Pending
  |
  |— completed
```

If the ACH takes two days, you need a durable Payment. There is nothing else to hold the state during those two days, and the invoice must not be marked paid before the funds arrive.

11.2 Partial payment against an invoice

```
Invoice  $10,000
├── Payment  $6,000 → Paid
└── Payment  $4,000 → Outstanding
```

THE ARCHITECTURAL REQUIREMENT

This is why the architecture must support **multiple Payments against an Invoice**, exactly as it supports multiple Payments against an Order. Invoice and Order play the same role: they state what is owed, and Payments collect it.

SECTION 12

Refunds and disputes

Payment gives both a stable parent, and both can occur long after the original collection.

12.1 Refund

```
Payment  $1,000
├── Capture  $1,000
├── Refund    $200
└── Refund    $300
```

WHAT NOT TO DO

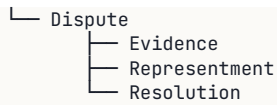
Do not set `Payment state = REFUNDED`. That loses the ability to represent multiple independent refund operations, and it collapses a money movement into a lifecycle state.

```
Payment
├── Capture
├── Refund
└── Refund
```

The financial truth lives in the Money and Ledger layer, not in a status field on the Payment.

12.2 Dispute

```
Payment  PAY-123
├── Attempt
├── Authorization
├── Capture
└── Refund
```

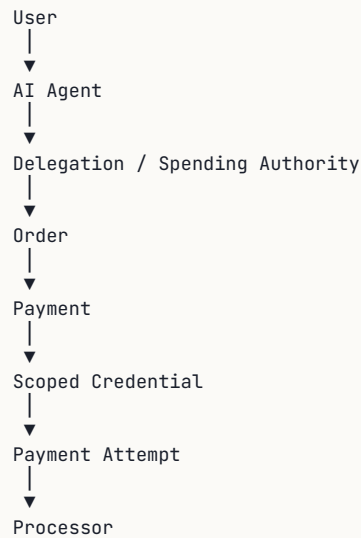


A dispute can arrive **weeks or months** after the original payment. The Payment therefore needs a durable identity that survives well past settlement — long after every Session has expired and every Attempt has closed.

SECTION 13

Agentic commerce

This becomes more important still, because authorization and execution are separated in time by design.



The agent may initiate the transaction, but execution can happen later. The Payment is the durable object linking every element of the chain of authority:

LINKED BY THE PAYMENT	LINKED BY THE PAYMENT
User authorization	Credential
Agent identity	Authentication
Merchant	Execution
Order	Money movement
Spending limit	

WHY THIS MATTERS DOWNSTREAM

When a dispute arrives on an agent-initiated transaction, this chain *is* the evidence. It exists only because the Payment held the objects together across a gap in time that no session and no attempt could span.

When Payment need not be exposed

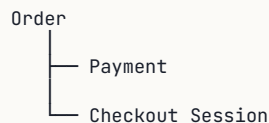
Equally important. The API must not force every developer to understand Payment.

14.1 A client with no order management system

They work at the Order, per Section 4.1.

```
POST /orders
POST /orders/{id}/checkout
```

The platform internally creates:



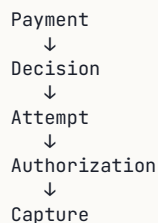
The merchant does not create a Payment manually, and never names one.

14.2 A client with an order management system

They work at the Payment, per Section 4.2, and carry their own order identifier.

```
POST /payments
{
  "amount": 10000,
  "currency": "USD",
  "order_ref": "WEB-2026-0884213",
  "payment_method": "pm_123",
  "confirm": true
}
```

Internally:



THE PRINCIPLE

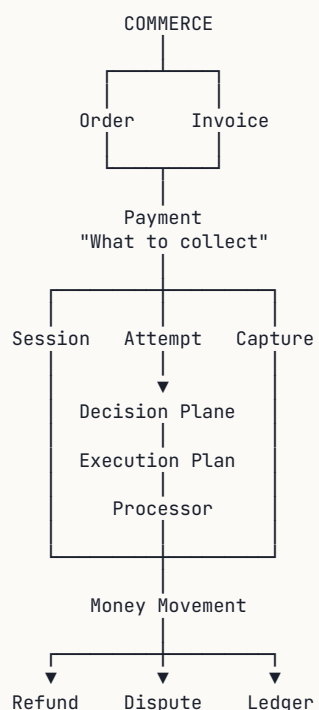
One API call. The complexity remains inside the platform.

Payment is a first-class canonical object internally, and progressively disclosed externally. Simple merchants experience "Order → Checkout → Paid." Sophisticated merchants control Payment → Attempt → Decision → Capture → Refund.

SECTION 15

The architecture to lock in

The fundamental relationship this document argues for.



THE SINGLE MOST IMPORTANT SEMANTIC DISTINCTION

Payment = durable collection obligation. Attempt = a particular execution of that payment.

That distinction is what lets the platform support e-commerce, recurring, invoices, multi-tender, omnichannel, card present, asynchronous rails, retries, routing, marketplaces, franchises and agentic commerce without creating a different payment model for every use case.

15.1 The two-line summary

AUDIENCE	WHAT THEY EXPERIENCE
No order system	Order → Checkout → Paid
Has an order system	Payment (+ order_ref) → Attempt → Decision → Capture → Refund

Same platform. Same objects. Same state machines. The only difference is which resource the client names — and that is decided by whether they already own an order record, not by what they are trying to do.